

Computer Organization & Architecture

1. Introduction

Computer Organization and Architecture is the study of how a computer system is designed, how its parts work together, and how instructions are executed efficiently. It explains the internal structure of a computer, including the processor, memory, input/output devices, control unit, buses, and performance techniques.

The subject is important because software runs on hardware, and understanding hardware structure helps in designing faster, more efficient, and more reliable systems. It also gives the foundation for topics such as assembly language, pipelining, cache memory, and computer performance.

2. Basic concepts

Computer organization describes the operational units and their interconnections, while computer architecture describes the attributes of a system visible to the programmer, such as instruction set, data formats, addressing modes, and memory model.

In simple terms, organization is about **how** the machine is built, and architecture is about **what** the machine offers to the programmer.

3. Functional blocks of a computer

A computer system consists of several major functional blocks: CPU, memory, input units, output units, and the system bus.

CPU

The Central Processing Unit is the brain of the computer. It contains the control unit, arithmetic and logic unit, and registers.

Memory

Memory stores instructions and data needed during execution. It includes registers, cache, main memory, and secondary storage in a hierarchy.

Input/Output subsystem

The I/O subsystem connects the computer with external devices such as keyboard, mouse, disk, printer, display, and network interfaces.

System bus

The bus carries data, address, and control signals between components.

4. CPU organization

The CPU is made up of three major parts: the control unit, arithmetic logic unit, and register set.

Control unit

The control unit coordinates all activities in the computer. It fetches instructions, decodes them, and generates control signals to carry out operations.

ALU

The Arithmetic Logic Unit performs arithmetic operations such as addition and subtraction, and logical operations such as AND, OR, XOR, and comparisons.

Registers

Registers are small, fast storage locations inside the CPU. Common registers include the program counter, instruction register, accumulator, stack pointer, and general-purpose registers.

5. Instruction set architecture

Instruction Set Architecture, or ISA, is the programmer-visible interface of the CPU. It defines the instructions, registers, addressing modes, data types, and execution model supported by the processor.

ISA is important because it determines what a machine can do and how software is written for it.

Major components of ISA

- Instruction format.
- Instruction set.
- Addressing modes.
- Register set.
- Data types.
- Exception and interrupt support.

6. Instruction cycle

The instruction cycle is the sequence of operations the CPU performs to execute a single instruction. It usually includes fetch, decode, execute, memory access, and write-back.

Steps

1. Fetch the instruction from memory.
2. Decode the instruction.
3. Fetch operands if needed.
4. Execute the operation.
5. Store the result.

7. Addressing modes

Addressing modes define how an instruction locates its operands. They make the instruction set more flexible and efficient.

Common addressing modes

- Immediate.
- Direct.
- Indirect.
- Register.
- Register indirect.
- Indexed.
- Base plus offset.
- Relative.

Importance

Addressing modes reduce instruction length, support arrays and pointers, and improve programming convenience.

8. Data representation

Computers store all data in binary form, so different representations are used for signed integers, floating-point numbers, and characters.

Signed number representation

Common signed number systems include sign-magnitude, 1's complement, and 2's complement. In practice, 2's complement is widely used because arithmetic is simpler.

Floating-point representation

Floating-point numbers represent very large or very small values using sign, exponent, and mantissa. This format is used for scientific and engineering calculations.

Character representation

Characters are represented using encoding schemes such as ASCII and Unicode.

9. Computer arithmetic

Computer arithmetic deals with how computers perform calculations in binary. It includes addition, subtraction, multiplication, division, and floating-point arithmetic.

Addition and subtraction

Binary addition is performed bit by bit, with carry handling. Subtraction is often done using 2's complement addition.

Multiplication

Common multiplication methods include shift-and-add, Booth's algorithm, and carry-save methods.

Division

Division can be performed using restoring and non-restoring methods.

Why these matters

Arithmetic units and algorithms determine processor speed and efficiency for numeric computation.

10. Control unit design

The control unit generates the control signals that coordinate CPU operations. There are two major design approaches: hardwired control and microprogrammed control.

Hardwired control

In hardwired control, control signals are generated by fixed logic circuits. It is fast but difficult to modify.

Microprogrammed control

In microprogrammed control, control signals are produced by executing microinstructions stored in control memory. It is more flexible but usually slower than hardwired control.

11. Memory system

The memory system stores instructions and data and is organized in a hierarchy based on speed, cost, and capacity.

Memory hierarchy

- Registers.
- Cache memory.
- Main memory.
- Secondary storage.

Why hierarchy matters

Faster memory is more expensive and smaller, so systems combine several memory levels to balance performance and cost.

12. Cache memory

Cache is a small, fast memory placed between the CPU and main memory to reduce the average memory access time. The CPU checks cache first before going to main memory.

Cache concepts

- Hit.
- Miss.
- Hit ratio.
- Block size.
- Replacement policy.
- Write policy.

Mapping methods

Common cache mapping methods include direct mapping, associative mapping, and set-associative mapping.

13. Main memory

Main memory usually refers to RAM, which stores active programs and data currently being used by the CPU. It must be fast enough to supply instructions and data quickly.

Types

- RAM.
- ROM.siiet.ac+4

14. I/O organization

Input/output organization describes how peripherals communicate with the CPU and memory. Common methods include programmed I/O, interrupt-driven I/O, and DMA.

Programmed I/O

The CPU continuously checks the status of the device. Simple but inefficient.

Interrupt-driven I/O

The device interrupts the CPU when it is ready, improving efficiency.

DMA

Direct Memory Access allows devices to transfer data to and from memory without continuous CPU intervention.

15. Interrupts and exceptions

Interrupts are signals that temporarily stop the current CPU activity so it can respond to an event. Exceptions are abnormal events detected during instruction execution.

Types of interrupts

- Hardware interrupts.
- Software interrupts.
- Maskable interrupts.
- Non-maskable interrupts.

Importance

Interrupts make multitasking, I/O handling, and process state transitions possible.

16. Pipelining

Pipelining improves CPU throughput by overlapping the execution of multiple instructions. Instead of waiting for one instruction to finish completely, the processor works on different stages of several instructions at the same time.

Pipeline stages

- Fetch.
- Decode.
- Execute.
- Memory access.
- Write-back.

Pipeline hazards

- Structural hazards.
- Data hazards.
- Control hazards.

Why it matters

Pipelining increases instruction throughput and is a major performance enhancement technique.

17. Performance metrics

Computer performance is measured using speed, throughput, latency, clock rate, CPI, and execution time.

Common terms

- Clock cycle time.
- Instructions per cycle.
- Total CPU time.
- Throughput.
- Speedup.

18. RISC and CISC

RISC systems use a small, simple instruction set designed for efficient execution, while CISC systems use a larger and more complex instruction set.

RISC

- Simple instructions.
- Often faster pipeline execution.
- Load/store design.

CISC

- More complex instructions.
- Fewer instructions may be needed for a task.
- Often more compact code.

19. Summary

Computer Organization & Architecture explains how computer systems are built and how they execute instructions efficiently. It connects the theory of hardware design with practical performance techniques such as caching, pipelining, and interrupt-based I/O. A strong understanding of this subject helps in both exams and advanced computing topics.